



Spark Configuration Core API (RDD)

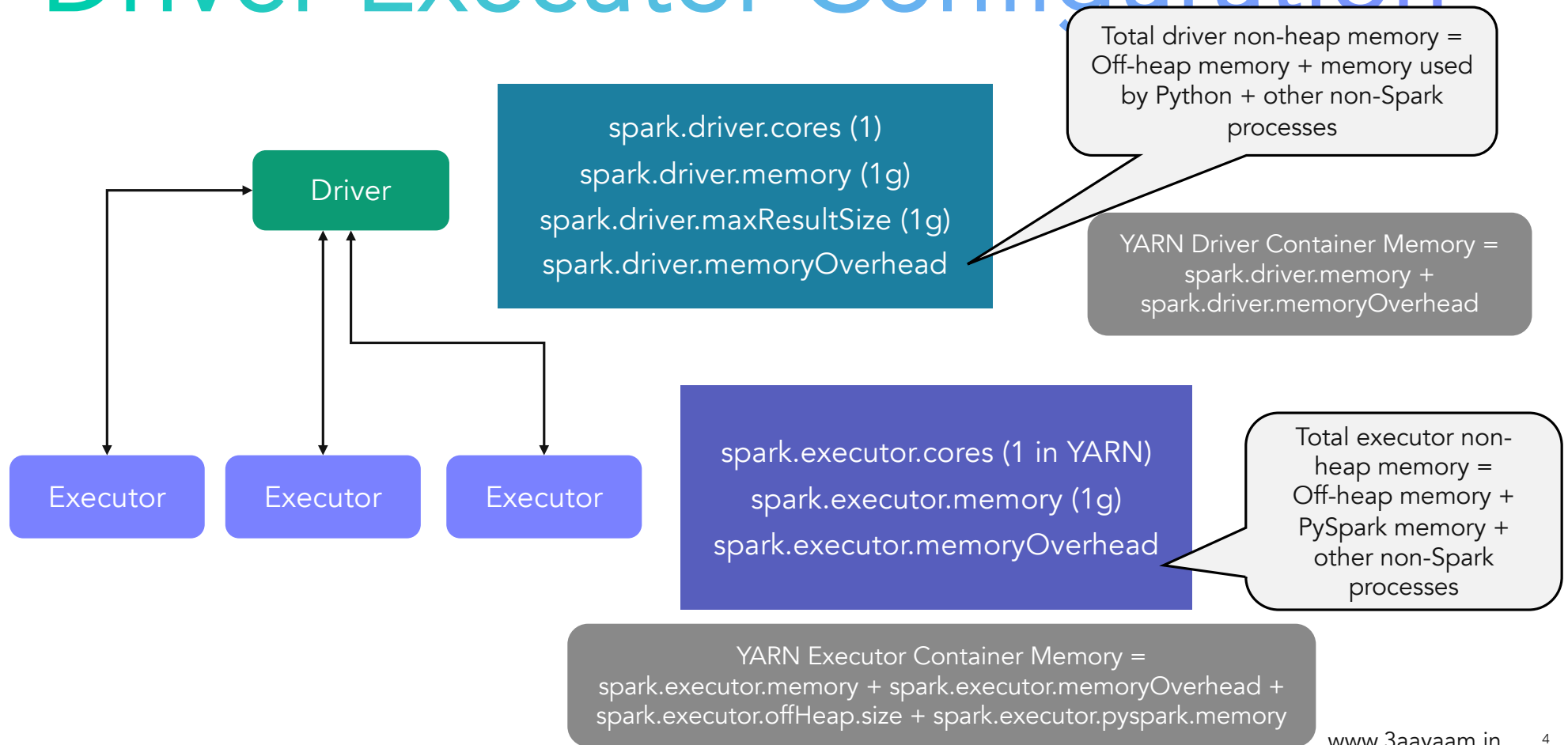
www.3aayaam.in

Agenda

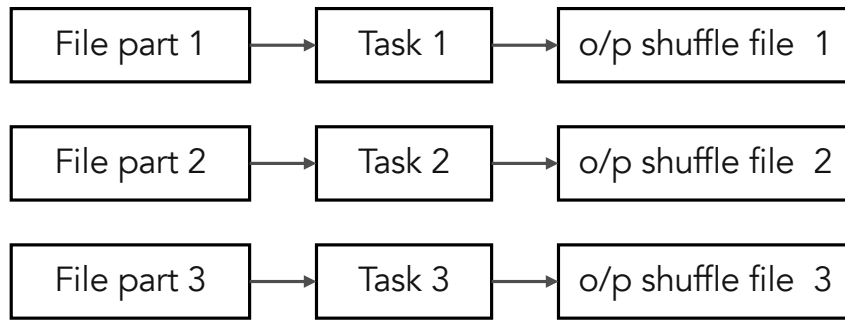
- Important Spark Configuration Parameters
- Spark RDD API High Level Introduction

Spark Configuration

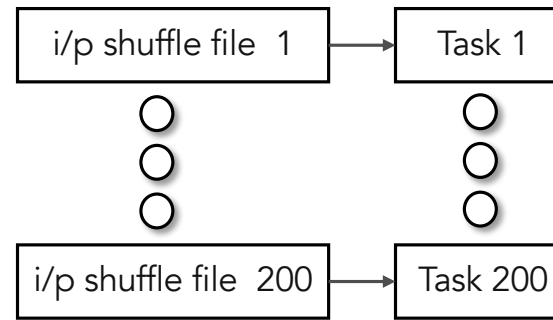
Driver Executor Configuration



Shuffle Configuration

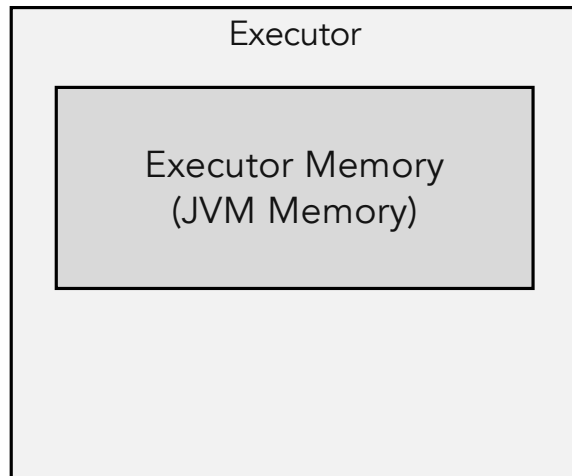


`spark.shuffle.service.enabled (false)`
`spark.shuffle.compress (true)`
`spark.shuffle.file.buffer (32k)`
`spark.shuffle.spill.compress (true)`



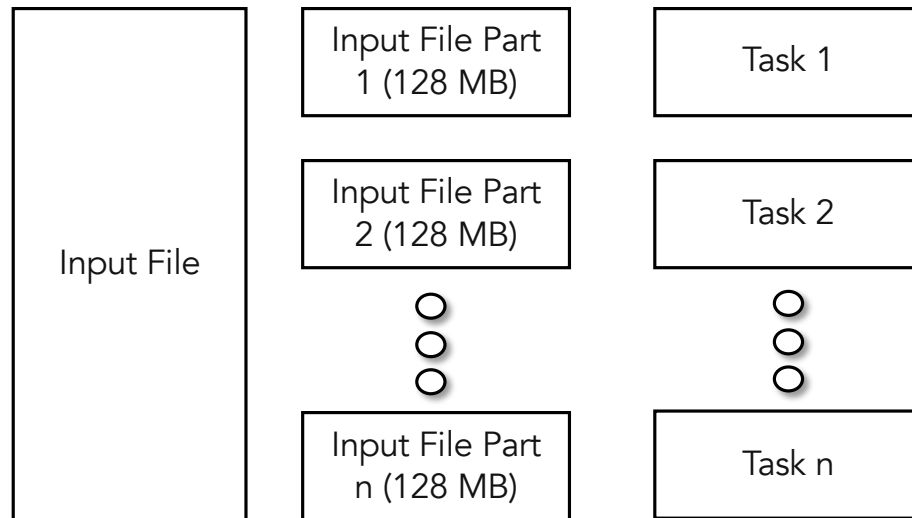
`spark.sql.shuffle.partitions (200)`

Memory Configuration



```
spark.executor.memory (1g)
spark.executor.memoryOverhead
spark.executor.memoryOverheadFactor (0.1)
spark.memory.fraction (0.6)
spark.memory.storageFraction (0.5)
spark.memory.offHeap.enabled (false)
spark.memory.offHeap.size (0)
```

Processing Configuration



`spark.default.parallelism` (total cores of executor nodes)

`spark.files.maxPartitionBytes` (128 mb)

`spark.scheduler.mode` (FIFO or FAIR)

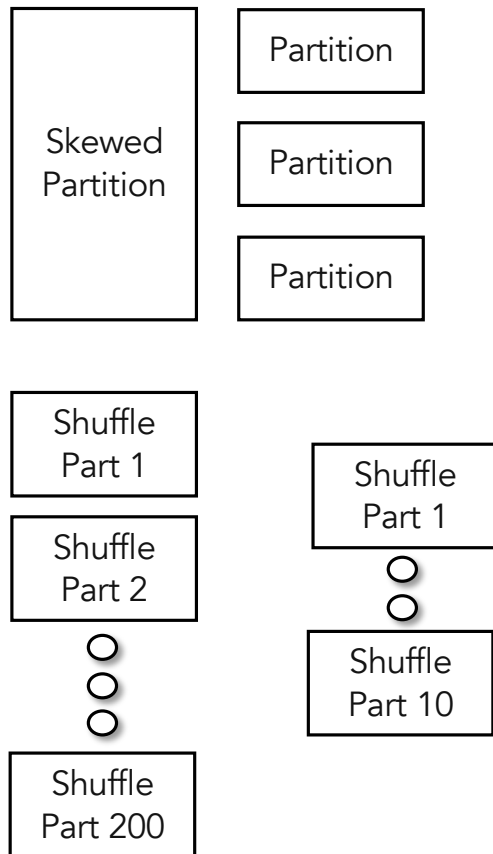
`spark.task.cpus` (1)

`spark.kryo.unsafe` (true)

`spark.kryoserializer.buffer.max` (64 mb)

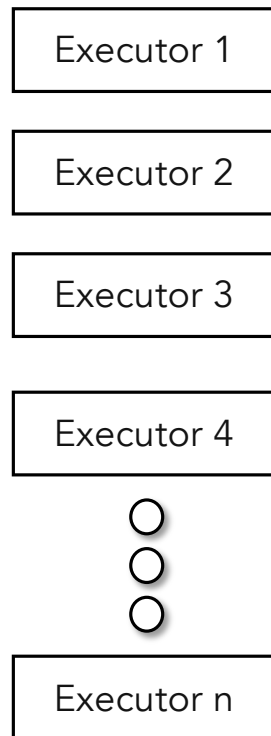
`spark.kryoserializer.buffer`

Adaptive Query Execution



```
spark.sql.adaptive.enabled
spark.sql.adaptive.advisoryPartitionSizeInBytes
spark.sql.adaptive.autoBroadcastJoinThreshold
spark.sql.adaptive.coalescePartitions.enabled
spark.sql.adaptive.coalescePartitions.initialPartitionNum
spark.sql.adaptive.coalescePartitions.minPartitionSize
spark.sql.adaptive.coalescePartitions.parallelismFirst
spark.sql.adaptive.skewJoin.enabled
spark.sql.adaptive.skewJoin.skewedPartitionFactor
spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes
```

Dynamic Resource Allocation



```
spark.dynamicAllocation.enabled  
spark.dynamicAllocation.initialExecutors  
spark.dynamicAllocation.minExecutors  
spark.dynamicAllocation.maxExecutors  
spark.dynamicAllocation.schedulerBacklogTimeout  
spark.dynamicAllocation.executorIdleTimeout  
spark.dynamicAllocation.cachedExecutorIdleTimeout
```

Spark Joins

Broadcast Hash Join

Broadcast Nested
Loop Join

Shuffled Nested
Loop Join

Shuffled Hash Join

Sort Merge Join

```
spark.sql.shuffledHashJoinFactor  
spark.sql.autoBroadcastJoinThreshold  
spark.sql.join.preferSortMergeJoin
```

Other Configs

```
spark.sql.autoBroadcastJoinThreshold (10 mb)
spark.sql.files.maxRecordsPerFile (0 = No Limit)
  spark.sql.json.filterPushdown.enabled (true)
  spark.sql.maxSinglePartitionBytes (128 mb)
  spark.sql.parquet.aggregatePushdown (false)
spark.sql.parquet.columnarReaderBatchSize (4096)
  spark.sql.parquet.filterPushdown (true)
  spark.sql.parquet.mergeSchema (false)
spark.sql.parquet.recordLevelFilter.enabled (false)
  spark.sql.shuffle.partitions (200)
```

Set & Get Config Parameters

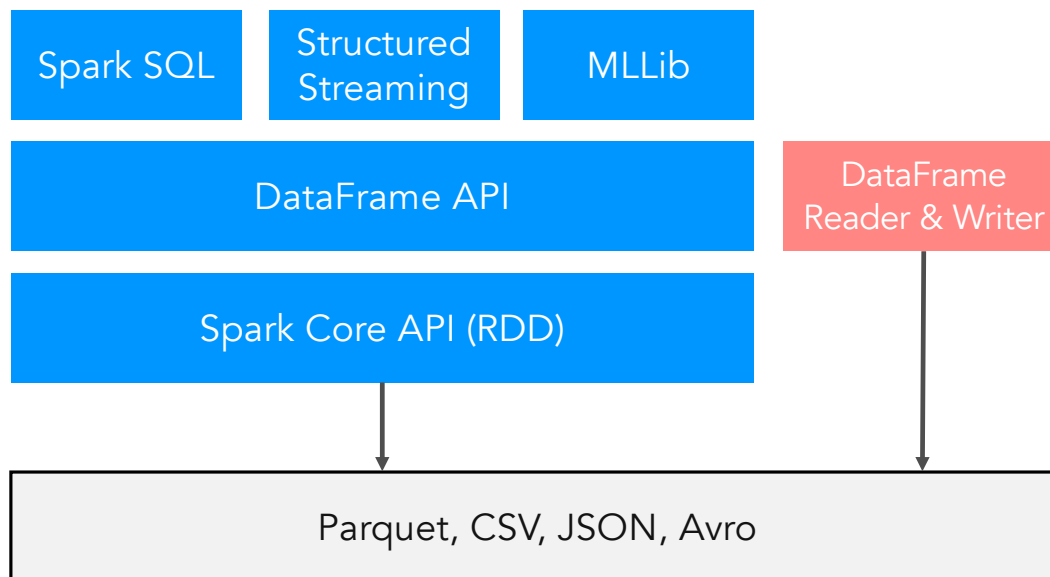
- `spark.get.conf("<config-parameter>")`
- `spark.set.conf("<config-parameter>", "<value>")`

www.3aayaam.in

Spark Core API (RDD)

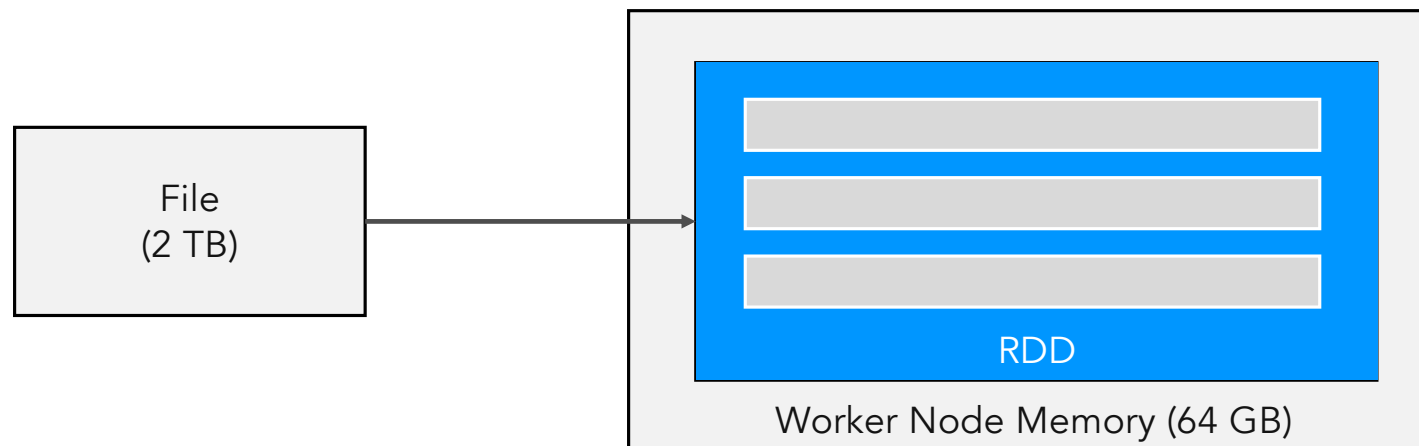
Spark Ecosystem

- Resilient Distributed Dataset.
- Stored in memory, Immutable, Distributed, Lazy Evaluation.
- No Structure or Schema.



What is RDD

- Resilient Distributed Dataset.
- Stored in memory, Immutable, Distributed, Lazy Evaluation.
- No Structure or Schema. Unstructured Data.

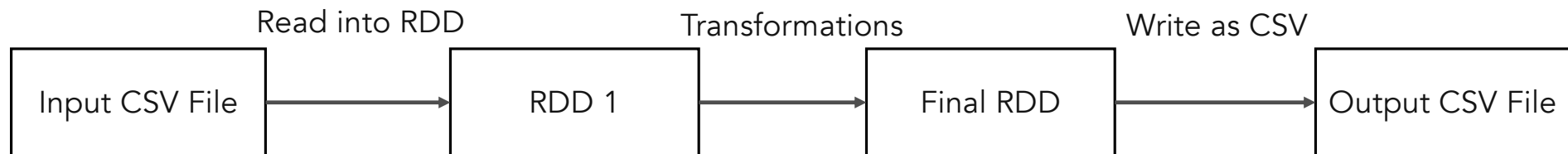


SparkContext

- It is similar to SparkSession, but does not support SQLs or DataFrames.
- Documentation: “Only one SparkContext per Executor JVM”. One SparkContext per driver, which is true for SparkSession also.
- SparkSession uses SparkContext underneath.

```
from pyspark import SparkContext, SparkConf
conf = SparkConf().setAppName().setMaster()
sc = SparkContext.getOrCreate(conf=conf)
```

textFile, saveAsTextFile



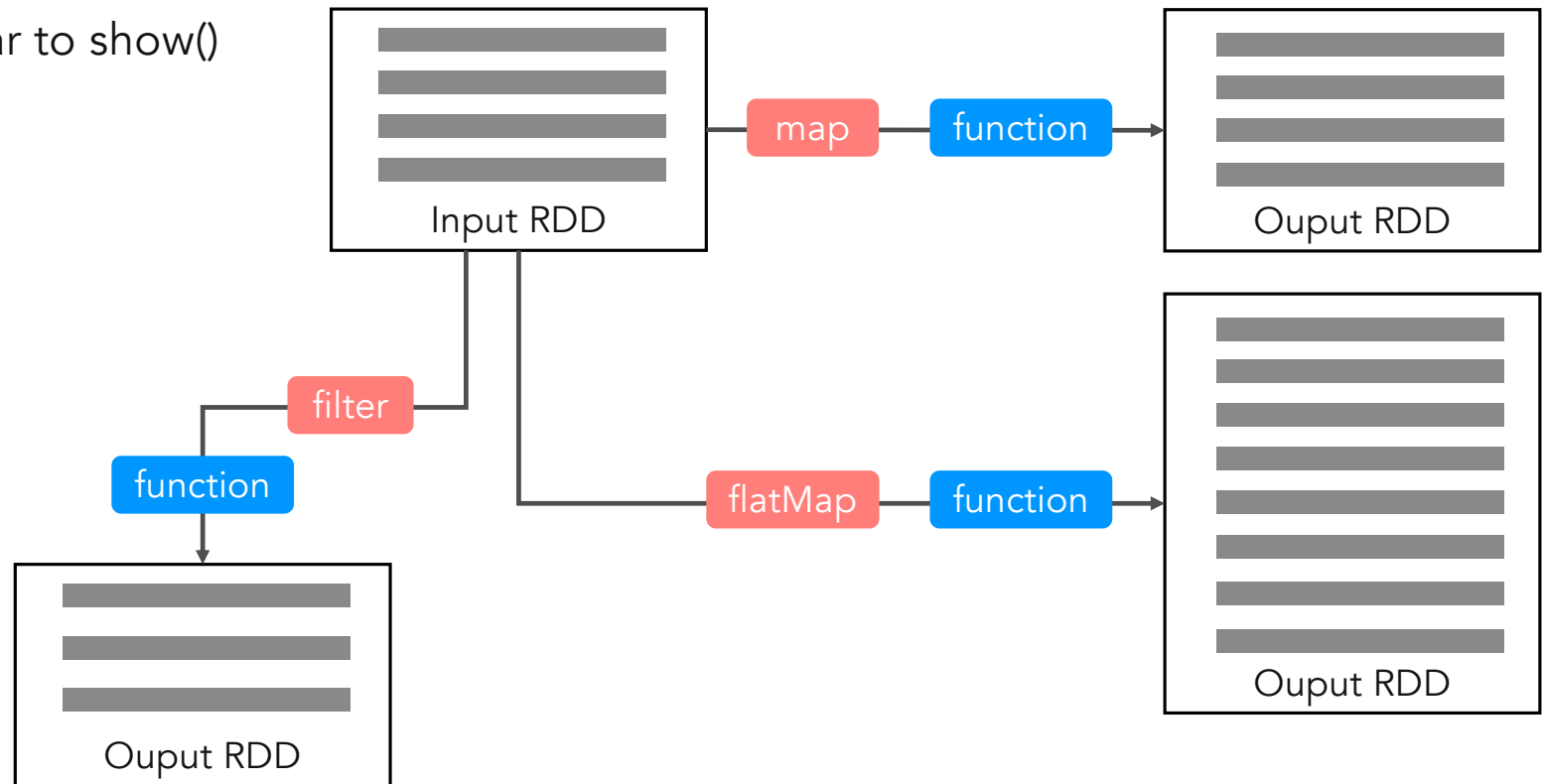
```
rdd1 = sc.textFile(<csv-file-location>)  
final_rdd.saveAsTextFile(<output-location>)
```

- Read and Write CSV files using RDD.
- For Parquet, Avro, JSON files – Read as DataFrame, convert them to RDD.
- SQLContext can be used to read and write Parquet, ORC, JSON, JDBC source & target.

```
sqlc = SQLContext(sc)
```

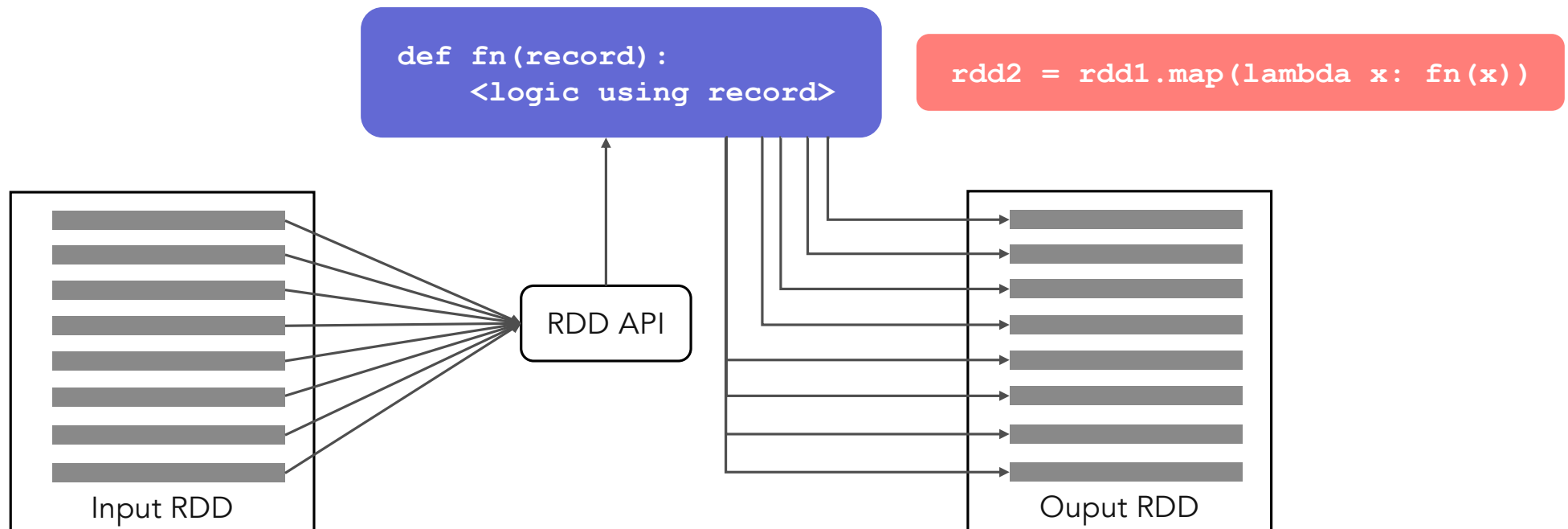
APIs – collect, map, flatMap, filter

- Collect – similar to show()
- map()
- flatMap()
- filter()



Function??

- User written Python function to apply transformation.
- How the records would be passed to the function.



RDD API HandsOn

- Implement DataFrame "Select" and "Filter/Where" APIs.

Other APIs

- `persist(StorageLevel.MEMORY_AND_DISK)`
- `keyBy`, `groupByKey`
- `cache()`
- `coalesce(n)` - Control output partitions.
- `partitionBy()`
- `groupBy()`, `sortBy()`, `sortByKey()`, `groupByKey()`, `reduce()`, `reduceByKey()`
- `count()`, `countByKey()`, `max()`, `min()`
- `aggregate()`, `aggregateByKey()`
- `distinct()`
- `join()`, `union()`, `rightOuterJoin()`, `leftOuterJoin()`
- ... etc.

Don't use RDD!!!
ALWAYS use DataFrame APIs

Summary

www.3aayaam.in

- Configuration for Driver & Executor
- Shuffle Configuration
- Memory Configuration
- Processing Configuration
- AQE Parameters
- DRA Parameters
- Join Configurations etc.
- RDD, SparkContext, SQLContext
- Read & Write Files.
- APIs – map, flatMap, filter, functions
- DON'T USE RDD, USE DATAFRAME



Thank You

www.3aayaam.in